

DriftConditions Implementation Plan

Spring 2026 Player Upgrade

1. Purpose

This plan covers improvements to the DriftConditions listener experience, centered on the audio player. It addresses a known multi-tab bug, introduces enhanced listening modes (fullscreen, sleep timer), adds basic social engagement features (share, favorites), and scopes out larger ambitions (casting, app distribution, platform reach). This is a living document and will be updated as implementation details emerge.

2. Goals and Scope

2.1 Goals

1. Fix multi-tab playback coordination — resolve the bug where stopping playback in one window triggers playback in another.
2. Honor user intent — play/stop state should reflect the listener's last deliberate action, regardless of how many tabs or windows are open.
3. Enhanced listening modes — fullscreen and sleep timer for a more intentional listening experience.
4. Mobile-first polish — all player features must work gracefully on mobile as well as desktop.
5. Engagement and sharing — let listeners favorite a mix and share what they're hearing.
6. Platform reach — investigate and implement distribution beyond the web player (cast, app, streaming platforms).

2.2 Why these goals matter

The current player is functional but fragile. The multi-tab bug actively undermines the listener experience. Enhanced modes (fullscreen, sleep) bring DriftConditions closer to how people actually use ambient audio. Sharing and favoriting add lightweight social hooks that benefit contributors and help grow the audience.

2.3 Scope

2.3.1 Player Core & Multi-tab Behavior

- Fix the inverted logic bug in cross-tab playback coordination.
- Redesign the shared-intent model so user actions propagate correctly across tabs/windows.

2.3.2 Enhanced Listening Experience

- Fullscreen mode: dark overlay, current mix graphic and playlist, dismissable.
- Sleep mode: auto-stop after a user-set timer.
- Media Session API integration for OS-level controls (lock screen, headphone buttons).

2.3.3 Engagement and Sharing

- Share button for the current mix.
- Favorite/star a mix.

2.3.4 Platform Reach (including Stretch Goals)

- Investigate podcast/radio directory listings (TuneIn, etc.).
- Cast to devices (Chromecast, AirPlay).
- PWA / App.
- Stream to YouTube Live.

2.4 Out of Scope

- Changes to MixEngine, recipe authoring, or audio moderation workflows.
- Any redesign of the contributor or admin interfaces.

3. Implementation: Player Core & Multi-tab Behavior

3.1. Multi-Tab Playback Coordination

The DriftConditions player needs to work correctly when the listener has multiple tabs open. The design centers on a single principle: **user intent is global**. When the listener presses play or stop, that action applies everywhere — not just in the tab where they clicked.

3.1.1. Use Cases

#	Scenario	Expected behavior
0	User opens the station page	Player shows stopped . No audio starts.
1	User presses play (single tab)	Audio starts. Player shows playing .
2	User presses stop (single tab)	Audio stops. Player shows stopped .
3	User presses play (multiple tabs open)	Audio plays in that tab. All other tabs update their display to show playing .
4	User presses stop in any tab (multiple tabs open)	Audio stops. All tabs show stopped .
5	User has multiple tabs open and closes a tab that shows playing but isn't the audio source	No effect. The playing tab continues uninterrupted.
6	User has multiple tabs open and closes the tab that is actually playing audio	Playback transfers to another open station tab.
7	User closes the last playing tab, then reopens the station	Player shows stopped . Audio does not auto-start.

3.1.2. Design

At any moment, at most one tab has the `<audio>` element actively playing. All other open tabs are silent, but their player UI mirrors the global intent — showing playing or stopped to match. When the user acts in any tab, that intent is broadcast to the rest.

3.1.2.1. Transport

Cross-tab coordination uses the `BroadcastChannel` API (channel name: `driftconditions-player`). This is purpose-built for same-origin tab messaging: real-time, no polling, and messages are not echoed back to the sending tab.

Four message types carry all necessary coordination:

type	Posted when	What other tabs do
play	Audio starts in this tab	Update display to playing ; silence their own audio

<code>stop</code>	User stops in this tab	Update display to <code>stopped</code> ; silence their own audio
<code>transfer</code>	This tab is closing while <code>playing</code>	Schedule a play attempt to take over
<code>query</code>	This tab just opened	Active player (if any) responds with <code>play</code> . Update display to <code>playing</code>

All messages include the sender's tab `id` so a tab can identify and correctly handle its own responses.

3.1.2.2. Initial State on Open

When a new tab opens, it posts a `query`. If another tab is currently playing, it responds with `play` and the new tab updates its display — but does not start audio. If no tab responds, the player stays stopped. This satisfies use cases 0 and 7 with no special-casing.

3.1.2.3. Tab Handoff on Close

When the playing tab closes, it posts `transfer` in a `beforeunload` handler before the page tears down. All other open tabs receive this and each schedules a play attempt after a short random delay (0–150ms). The first tab to start playing posts `play`, which causes all others to cancel their pending timers and stand down.

3.1.3. Component Design

`AudioPlayer.js` is the only file that changes. The following refs are added:

Ref	Purpose
<code>channelRef</code>	The <code>BroadcastChannel</code> instance
<code>isActivePlayerRef</code>	Whether this tab is the audio source
<code>suppressRef</code>	Prevents feedback: stops <code>handlePause</code> from broadcasting when audio is silenced programmatically
<code>transferTimeoutRef</code>	Holds the cancellable handoff timer

3.1.3.1. Event Handlers

`handleStart` — marks this tab as the active player; posts `play`.

`handlePause` — if `suppressRef` is set, clears it and returns without broadcasting (the pause was programmatic, not user-initiated). Otherwise, marks this tab as inactive and posts `stop`.

3.1.3.2. Effects

Channel listener — responds to incoming messages:

- **play** → cancel any pending handoff timer; suppress-pause local audio; show playing state
- **stop** → cancel any pending handoff timer; suppress-pause local audio; show stopped state
- **transfer** → schedule a play attempt after random 0–150ms jitter
- **query** → if this tab is the active player, respond with **play**

beforeunload handler — if this tab is the active player, posts **transfer** and sets **suppressRef** so the browser-triggered pause during teardown does not also post **stop**.

Mount effect — posts **query** to establish initial display state.

3.1.4. Considerations

Safari compatibility. **BroadcastChannel** requires Safari 15.4+. If older Safari support is needed, a **localStorage** fallback can be layered in without restructuring the logic — write intent to a dedicated key on **play/stop**, and treat incoming **storage** events the same as channel messages.

Handoff on mobile. The transfer path calls **audioRef.current.play()** programmatically, which mobile browsers may block without an active gesture context. Tab handoff is primarily a desktop pattern; this limitation is acceptable.

Error and stream-end behavior. **handleError** and **handleEnded** both call **audioRestart()**, which remounts the audio element. Neither posts to the channel — an error or stream dropout in one tab does not affect other tabs' display state or playback.

3.3 Mobile Behavior

The native **<audio>** element with **autoPlay** is blocked on mobile browsers (requires a user gesture). The current player already handles this in practice because playback is triggered by a tap (the faux player on the homepage). Verify this holds across the enhanced modes added in section 3.

3.4 Stale Buffer on Resume

- **Problem statement** — **.pause()** retains the buffer; a resume hours later replays stale cached bytes before eventually re-syncing with live

- **Implementation** — `stoppedAt = Date.now()` on stop; threshold check on resume (30 min suggested); if stale, reset `src → '' → load() → url` before `.play()`; reset `stoppedAt` on resume to avoid mid-session false triggers
- **BroadcastChannel interaction** — `stoppedAt` included in the stop broadcast so all tabs share the same reference; any tab resuming after 30+ min treats it as a cold start
- **Mobile** — same `src` reset works on iOS; no autoplay issue since it's inside a user gesture handler
- **Open decision** — configurable threshold vs. hardcoded 30 min

4. Implementation: Enhanced Listening Experience

4.1 Fullscreen Mode

A fullscreen listening mode: the screen goes dark, the current mix cover image fills the view, and the playlist is displayed below or overlaid. Dismissible by tap/click or Escape key.

Implementation approach:

- Use the [Fullscreen API](#) (`element.requestFullscreen()`) triggered from a button in the player UI.
- A new `FullscreenPlayer` component (or overlay within `AudioPlayer.js`) renders the cover image from Redux (`state.queue.currentMix.coverImage`) and the playlist from `state.queue.playlist`.
- On mobile, "fullscreen" may mean a full-page overlay rather than the native fullscreen API, since mobile browsers restrict it.

Open decision: Does fullscreen include playback controls (play/pause, stop), or just the display? Should the playlist be scrollable or just show the current mix?

4.2 Sleep Mode

Auto-stop after a user-set countdown. Common use: falling asleep to ambient audio.

Implementation approach:

- A sleep timer UI (e.g., dropdown: 15 min / 30 min / 1 hr / custom).
- A countdown stored in component state; when it reaches zero, call `audioRef.current.pause()` and broadcast the stop intent.
- Optional: fade out over the last 60 seconds rather than hard stop.

Open decision: Should the sleep timer persist across page refreshes (stored in `localStorage`), or reset on navigation?

4.3 Media Session API

The Media Session API lets the OS display playback info on the lock screen and respond to hardware controls (headphone buttons, car displays).

Implementation: Set `navigator.mediaSession.metadata` with the current mix title and cover art when a new mix starts. Wire `navigator.mediaSession.setActionHandler` for play, pause, and stop.

This is a small addition with high perceived quality on mobile.

5. Implementation: Engagement and Sharing

5.1 Share

Let listeners share what they're hearing.

What to share: A link to the DriftConditions homepage (since the stream is live and continuous, there's no permalink to a specific mix). The share payload would be the site URL with a short description of the current mix title.

Implementation:

- Use the [Web Share API](#) on mobile (native share sheet).
- Fall back to a copy-to-clipboard button on desktop.
- Share text: "Listening to [mix title] on DriftConditions – [url]".

Open decision: Should sharing include the current playlist details, or just a generic site link?

5.2 Favorite / Star a Mix

Let listeners mark a mix as a favorite while it's playing.

Implementation:

- A star/heart button in the player UI, visible when a mix is playing.
- On tap: sends a request to a new `POST /api/queue/favorite` endpoint (or similar) with the current `mixID`.
- Requires a new `favorites` table in the DB (or a `favorites` column on `mixQueue`), keyed by `userID + mixID`.
- For unauthenticated users: either prompt to sign in, or store anonymously (to be decided).

- Favorited mixes could eventually surface in user profiles or inform recipe weighting.

Open decision: Is favoriting limited to logged-in users? If so, should there be an incentive to sign up to favorite? What, if anything, does a favorite do for contributors or the system?

6. Stretch Goals

6.1 PWA / Installable App

A Progressive Web App makes the site installable on iOS and Android home screens, with offline support and a native-feeling experience.

What it requires: A web app manifest (manifest.json) and a service worker. React's build tooling (Create React App / Vite) makes this relatively low effort. Most of the enhanced features above (fullscreen, sleep, media session, share) already work as a PWA.

Decision point: Native iOS/Android apps are a separate, much larger project. PWA is the recommended starting point unless native-only features (e.g., background audio on iOS without a browser) are required.

6.2 Cast to Devices

Chromecast (via the Cast SDK) and AirPlay (via `<audio>` with Safari's native controls or WebKit's `x-webkit-airplay`) let listeners send audio to speakers or TVs.

AirPlay requires essentially nothing on our end — Safari exposes an AirPlay button natively on the `<audio>` element. Chromecast requires integration with the Cast SDK and a registered Cast app.

Recommendation: Enable AirPlay first (zero-cost, just verify it works). Evaluate Chromecast as a stretch.

6.3 Stream to YouTube Live

Re-stream the existing Icecast output to YouTube Live via RTMP. This is a server-side operation — ffmpeg can read the Icecast stream and push it to YouTube's ingest URL.

C/C++

```
ffmpeg -i https://driftconditions.org/stream \

-c:a aac -b:a 128k \

-f flv rtmp://a.rtmp.youtube.com/live2/[stream-key]
```

This could be a systemd service alongside MixEngine. No client changes required.

Open questions: Does YouTube require a video track for live streams? (Yes — a static image or visualizer would be needed.) Is 24/7 streaming permitted under YouTube's terms for this content type?

6.4 Podcast and Radio Directory Listings

Traditional podcast apps (Apple Podcasts, Spotify Podcasts) expect on-demand episodes with an RSS feed — a poor fit for a continuous generative stream.

Better fits:

- TuneIn / iHeartRadio — internet radio directories that accept Icecast stream URLs. Low barrier, broad reach.
- Overcast / Pocket Casts — some podcast apps support live radio streams, but discoverability is poor.

Recommendation: Submit to TuneIn as a radio station. Investigate requirements. Skip traditional podcast directories unless an archiving/episodes feature is added later.

7. Risks

Risk	Likelihood	Impact	Mitigation
BroadcastChannel not supported in older Safari	Low	Medium	<code>localStorage</code> fallback
Mobile autoplay restrictions break fullscreen mode	Medium	Medium	Ensure fullscreen is only triggered by a user gesture

Fullscreen API unavailable on some mobile browsers	Medium	Low	Fallback to full-page CSS overlay
YouTube 24/7 streaming policy violations	Unknown	High	Review ToS before implementing
Favoriting with anonymous users creates spam/abuse vector	Medium	Medium	Require login, or rate-limit by IP

8. Verification

- Multi-tab fix: Open 3 tabs. Play in tab 1, stop in tab 2 — confirm all tabs stop. Play in tab 3 — confirm all tabs play. Open a new tab mid-playback — confirm expected behavior (per open decision above).
- Fullscreen: Verify on mobile (iOS Safari, Android Chrome) and desktop. Confirm cover image and playlist render correctly. Confirm dismiss works.
- Sleep mode: Set a 1-minute timer. Confirm audio stops without hard cutoff. Confirm timer resets on page reload.
- Share: Verify Web Share API fires on mobile. Verify clipboard fallback on desktop.
- Favorite: Confirm star persists after page reload (if logged in). Confirm unauthenticated behavior matches decision.
- Media Session: Verify lock screen controls on iOS and Android.

9. Appendices

9.1 Relevant Files

File	Role
<code>AdminClient/src/components/AudioPlayer.js</code>	Core player component; <code><audio></code> element, cross-tab coordination

<code>AdminClient/src/layouts/RootLayout.js</code>	Mounts <code>AudioPlayer</code> ; owns <code>isPlaying</code> state; passes <code>togglePlayer</code> via context
<code>AdminClient/src/pages/Homepage.js</code>	Renders the faux-player button; consumes <code>togglePlayer</code> from outlet context
<code>AdminClient/src/components/Playlist.js</code>	Playlist display component; polls <code>MixEngine</code> every 60s
<code>AdminClient/src/store/queueSlice.js</code>	Redux slice; holds <code>playlist</code> and <code>currentMix</code> (<code>playlist[1]</code>)
<code>AdminClient/src/config/config.js</code>	<code>config.stream.url</code> — the Icecast/FastCast stream URL

9.2 Open Decisions

1. New-tab autoplay: Should a new tab auto-start if the global intent is "playing"?
2. Fullscreen controls: Play/pause included in fullscreen view?
3. Sleep timer persistence: Survive page refresh or reset?
4. Share payload: Site URL only, or include current mix details?
5. Favoriting and auth: Logged-in only? What does a favorite do downstream?
6. App path: PWA first, or straight to native?
7. YouTube: Static image or visualizer for video track? Review streaming ToS.

9.3. Appendix E — Document Style Guide

This section details the standardized formatting applied to the document.

9.3.1. Headers and Section Titles

This style dictates the formatting for all section titles and headings (Heading 1, Heading 2, Heading 3, etc.) to ensure clear hierarchy and visual distinction from body text. All headers must be formatted exclusively using these standard Google Docs heading styles rather than custom text styles.

- **Font Family:** Arial
- **Font Size:** Varied (e.g., 20pt for Heading 1, 16pt for Heading 2, 14pt for Heading 3)

- **Font Weight:** Bold
- **Application:** Applied exclusively to section titles and subtitles, using the designated Docs Heading styles.

Structural Patterns: H1 headings (e.g., "1. Goals and Scope") must use the "Heading 1" doc style; H2 headings (e.g., "1.1. Goals") must use the "Heading 2" doc style; and this hierarchical pattern must be followed for all subsequent heading levels.

9.3.2. Body Paragraphs and Standard Prose

This section defines the baseline formatting for all narrative and descriptive text in the document. Standardized prose ensures optimal readability and consistency across all sections, distinguishing it clearly from headings and technical lists.

- **Font Family:** Arial
- **Font Size:** 11pt
- **Line Spacing:** 1.15
- **Spacing (Before Paragraph):** 0pt
- **Spacing (After Paragraph):** 12pt
- **Application:** Applied exclusively to normal body text, excluding headings, titles, and list items.

9.3.3. Inline Technical References

This style is used for all short code references embedded within the main text, such as variable names, file paths, API routes, and terminal commands. It serves to visually isolate code elements from surrounding narrative text.

- **Font Family:** Roboto Mono (Monospaced)
- **Text Color:** Gemini Green (#188038)
- **Background:** None
- **Use Cases:** Inline code snippets (e.g., `clipUsage` table), variable names (e.g., `audioID`), file paths (e.g., `config/config.js`), and console output text.

9.3.4. Code Block Syntax Highlighting Style: One Light (Notion variant)

Background: Light gray (#f7f7f7 or similar off-white) **Default text:** Dark gray (identifiers, punctuation, table/column names)

Token colors:

- **Keywords** (`ALTER`, `CREATE`, `ADD`, `NOT`, `NULL`, `ON`, `DELETE`): Medium blue (#0070c1 or similar)
- **Types** (`INT`, `VARCHAR`, `DATETIME`): Bright blue (#0099cc or similar)

- **Numeric literals** (255, 32, 191): Orange-red (#e06c00 or similar)
- **Comments** (-- ...): Muted pink/mauve (#c678a0 or similar)
- **Language label** (SQL): Small-caps, light gray, non-highlighted

Typography: Monospace, medium weight, line height of 1.15. Token colors are distinct but not garish — optimized for readability over visual drama.

```
SQL
-- Existing 'users' table update (only adding utility columns)
ALTER TABLE users
  ADD COLUMN displayName VARCHAR(255),
  ADD COLUMN lastLoginAt DATETIME;

-- New 'userIdentities' table to support multiple linked OAuth accounts
CREATE TABLE userIdentities (
  userId INT NOT NULL, -- Foreign key to the users table
  oauthProvider VARCHAR(32) NOT NULL,
  oauthId VARCHAR(191) NOT NULL,
  PRIMARY KEY (oauthProvider, oauthId),
  FOREIGN KEY (userId) REFERENCES users(userId) ON DELETE CASCADE
);

-- Allow null names since OAuth providers may not supply them
ALTER TABLE users
```